Bazaar Inventory Tracking System

Case Study Challenge | Engineering

By : Hammad Malik
University : Habib University

بازار

BAZAAR

Habib University
shaping futures

Overview

This system tracks product inventory and stock movements across multiple stores as given in the Case Study PDF, with support for real-time stock visibility, audit capabilities, and scalable architecture to handle thousands of stores.

Features

- **Track inventory:** Monitor stock levels across stores in real-time
- **Stock movements:** Record stock-in, sales, and manual removals
- **Multi-store support:** Central product catalog with store-specific inventory
- **Authentication & authorization:** Role-based access control with store-specific permissions
- **Reporting:** Low stock alerts, inventory valuation, and sales tracking
- **Audit:** Complete history of all inventory changes
- **Scalability:** Designed to handle thousands of stores and high transaction volumes

Bazaar Inventory Management System - v1

Stage 1: Single Store

SQLite Database: Local storage for a single store

- Embedded relational database stored on disk (inventory_stage1.db)
- Manages product information, inventory levels, and stock movement history
- Ideal for lightweight, single-user environments

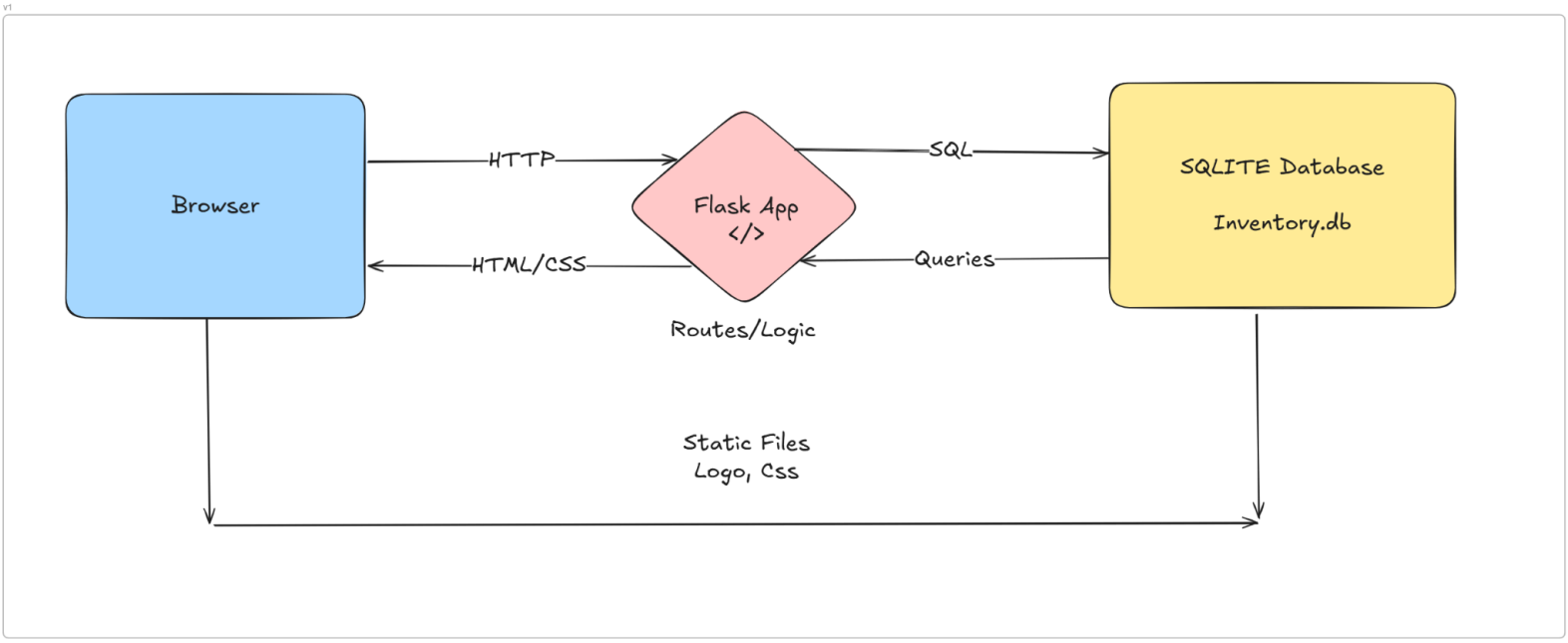
Backend On Flask

- Implemented using Python (Flask)
- Adding and retrieving products
- Recording inventory actions (stock-in, sale, removal)
- Responsible for database initialization, validation, and transaction integrity

User Interface

- Web based interface for easy-to-understand UI
- User-friendly forms for product management
- Visual dashboard for live inventory overview
- Bootstrap-enhanced for responsiveness

System Architecture Diagram for v1:



Bazaar Inventory Management System - v2

Design Decisions

Multi-Store Architecture

- Central Product Catalog: All stores share a common product database, enabling consistent product information across the organization
- Store-Specific Inventory: Each store maintains separate inventory records, allowing for location-specific stock management
- Event-Based Stock Movement: All inventory changes are tracked as events with timestamps, users, and references for complete audit trail

Database Design

- Relational Database: Chose PostgreSQL for ACID compliance, transaction support, and better concurrency handling
- Normalization: Database schema follows proper normalization principles to avoid data redundancy
- Foreign Key Constraints: Enforces data integrity across related tables
- Timestamp Tracking: All entities include created_at and updated_at fields for audit and tracking

Security Implementation

- JWT Authentication: Secure, stateless authentication using JSON Web Tokens
- Role-Based Access Control: Three primary roles - admin, store_manager, and store_user

- Store-Based Permissions: Users have access only to their assigned stores (except admins)
- Password Hashing: Passwords are securely hashed and never stored in plain text
- Rate Limiting: API endpoints are protected against brute force and DoS attacks

Containerization Strategy

- Docker Isolation: Each component runs in its own container for better isolation and scalability
- Simplified Deployment: Docker Compose orchestration simplifies setup across environments
- Environment Consistency: Ensures consistent behavior across development and production
- Volume Persistence: Database data persists across container restarts

Key Assumptions

- Internet Connectivity: Assumes reliable internet connectivity for API communications between stores
- Data Volume: System can handle moderate transaction volumes (up to thousands per day per store)
- Concurrency: Multiple users may access the system simultaneously from different stores
- Transaction Size: Most inventory operations involve reasonable quantities (not millions of items in a single transaction)
- Product Uniqueness: Products are unique across the entire organization with a single central catalog

- User Access Patterns: Store users primarily access their own store's data, while admins need cross-store visibility

API Design

Design Principles

- RESTful Architecture: Clear resource-oriented design with appropriate HTTP methods
- Versioning: All endpoints prefixed with /api/v2/ to allow future version changes
- Consistent Response Format: Standardized success and error response structures
- Pagination: All list endpoints support pagination to handle large data sets
- Filtering: Flexible query parameters for filtering data
- Authentication: JWT-based with refresh token capability
- Documentation: Comprehensive API documentation with examples

Endpoint Structure

- Authentication Endpoints: /api/v2/auth/* for login, registration, token refresh
- Product Endpoints: /api/v2/products/* for central catalog management
- Inventory Endpoints: /api/v2/inventory/* for stock management
- Store Endpoints: /api/v2/stores/* for store management
- Reporting Endpoints: /api/v2/reports/* for analytics and insights

Security Measures

- Input Validation: All request data is validated before processing
- Rate Limiting: Prevents API abuse with per-endpoint limits
- Authorization Middleware: Enforces access control at the route level
- Secure Headers: Properly configured security headers

Evolution Rationale (v1 → v2)

Architectural Evolution

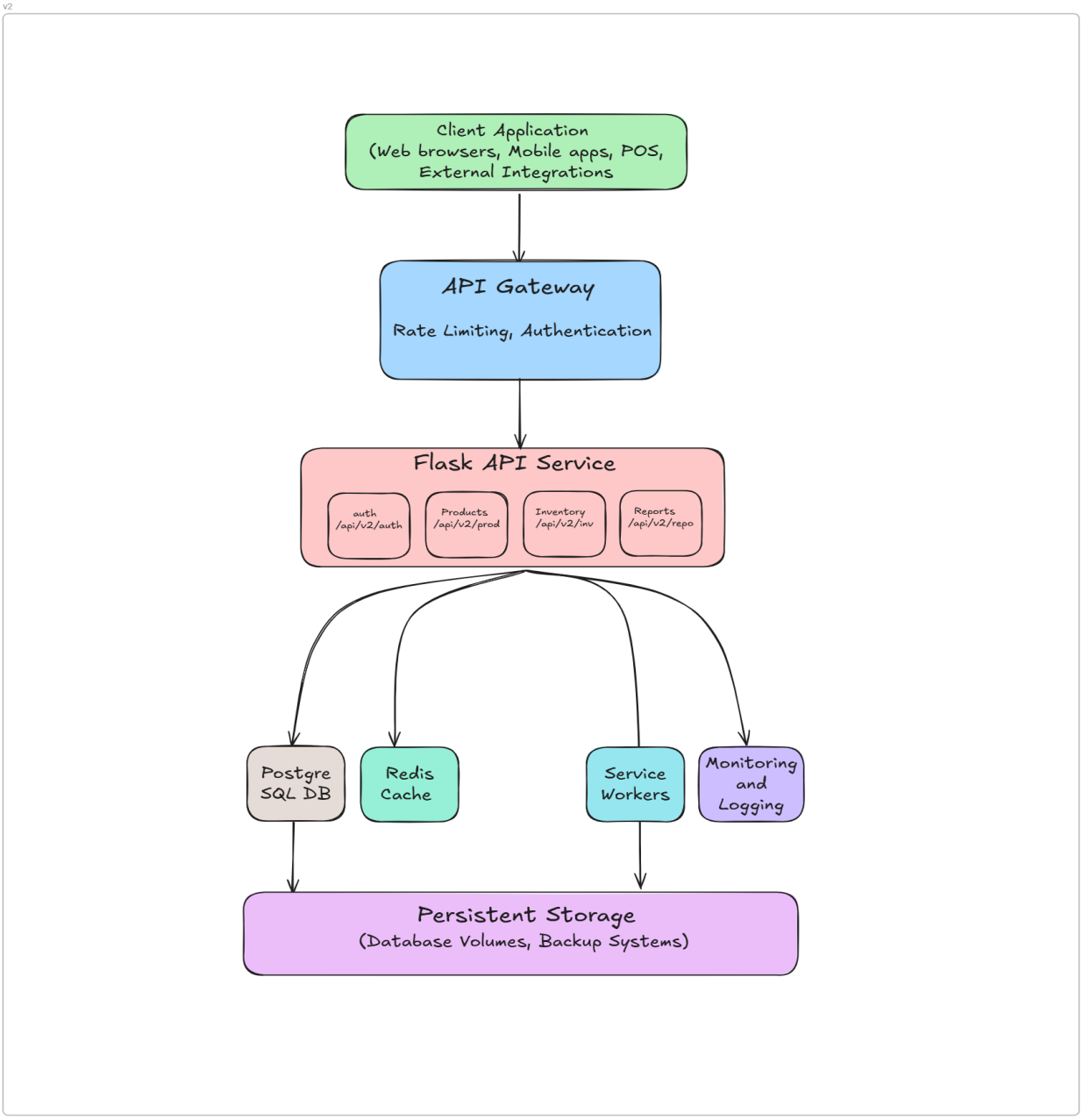
- From Monolith to API Service:
    - v1: Single-application monolith with direct HTML rendering
    - v2: API-first design with separation of concerns, enabling multiple front-end options
- From Single to Multi-Store:
    - v1: Limited to a single store's inventory
    - v2: Supports multiple stores sharing a central product catalog
- From Basic to Advanced Security:
    - v1: Simple session-based authentication
    - v2: JWT-based authentication with role-based access control
- From File-based to Relational Database:
    - v1: SQLite for simplicity
    - v2: PostgreSQL for scalability and concurrency

Technical Implementation Changes

- Database Migration:
    - Enhanced schema with store-related tables
    - Added foreign key relationships
    - Improved indexing strategy

- Authentication System:
    - Implemented JWT token-based authentication
    - Added token refresh mechanism
    - Created role-based authorization
- API Structure:
    - Organized endpoints into logical blueprints
    - Added versioning support
    - Implemented consistent response formatting
- Containerization:
    - Added Docker and Docker Compose
    - Created separate containers for API, database, and cache
    - Implemented volume persistence
- Performance Enhancements:
    - Added Redis for rate limiting and caching
    - Implemented database connection pooling
    - Optimized query patterns

System Architecture Diagram for v2:



Bazaar Inventory Management System v3

Design Decisions

Microservices Architecture

- Separated core functionality into independent components (API service, worker service)
- Used message queues for communication between services
- Implemented containerization for all components to enable independent scaling

Database Read/Write Separation

- Implemented primary/replica architecture to optimize performance
- Used a session manager to route read and write operations appropriately
- Added failover capability to maintain availability during replica downtime

Caching Strategy

- Implemented Redis-based caching for frequently accessed data
- Applied cache invalidation patterns to maintain data consistency
- Created cache management endpoints for administration

API Rate Limiting

- Implemented rate limiting based on client IP
- Used different rate limits for various endpoints based on impact
- Stored rate limit data in Redis for persistence across API instances

Load Balancing

- Incorporated Nginx as a reverse proxy and load balancer
- Implemented health checks to ensure traffic only goes to healthy instances
- Used least connections algorithm for optimal request distribution

Assumptions

Security Assumptions

- JWT tokens are sufficient for authentication and authorization
- Access control by store is required for multi-store environment
- Admin users need system-wide access to all stores

Data Assumptions

- Products have a central catalog shared across all stores
- Inventory is store-specific
- Stock movements must be trackable and auditable

Scaling Assumptions

- Read operations will significantly outnumber write operations
- Report generation may be resource-intensive and should be processed asynchronously
- The system must handle thousands of stores with minimal performance degradation

User Assumptions

- Multiple user roles are needed (admin, store manager, store user)
- Users will primarily access the API through a frontend application
- API will need to be consumed by both web and mobile clients

API Design

RESTful Principles

- Used consistent resource-based URL structure
- Implemented proper HTTP methods (GET, POST, PUT, DELETE)
- Applied consistent response formatting with appropriate status codes

Versioning Strategy

- Implemented URL-based versioning (v2 → v3)
- Maintained backward compatibility where possible
- Added new endpoints for enhanced functionality

Authentication and Authorization

- JWT-based authentication with separate login endpoint
- Role-based access control for different operations
- Store-specific access restrictions for non-admin users

Pagination and Filtering

- Implemented standard pagination for list endpoints
- Added filtering capabilities for complex data queries
- Included metadata in responses for client-side pagination handling

Evolution Rationale (v2 → v3)

Scalability Improvements

- Added horizontal scaling capabilities for all services

- Implemented database read/write separation for better performance
- Containerized all components with Docker Compose for easier deployment and scaling

Reliability Enhancements

- Added asynchronous processing via message queues
- Implemented background workers for resource-intensive operations
- Added health checks and failover mechanisms

Performance Optimizations

- Implemented Redis caching for frequently accessed data
- Added database connection pooling
- Optimized database queries with proper indexing

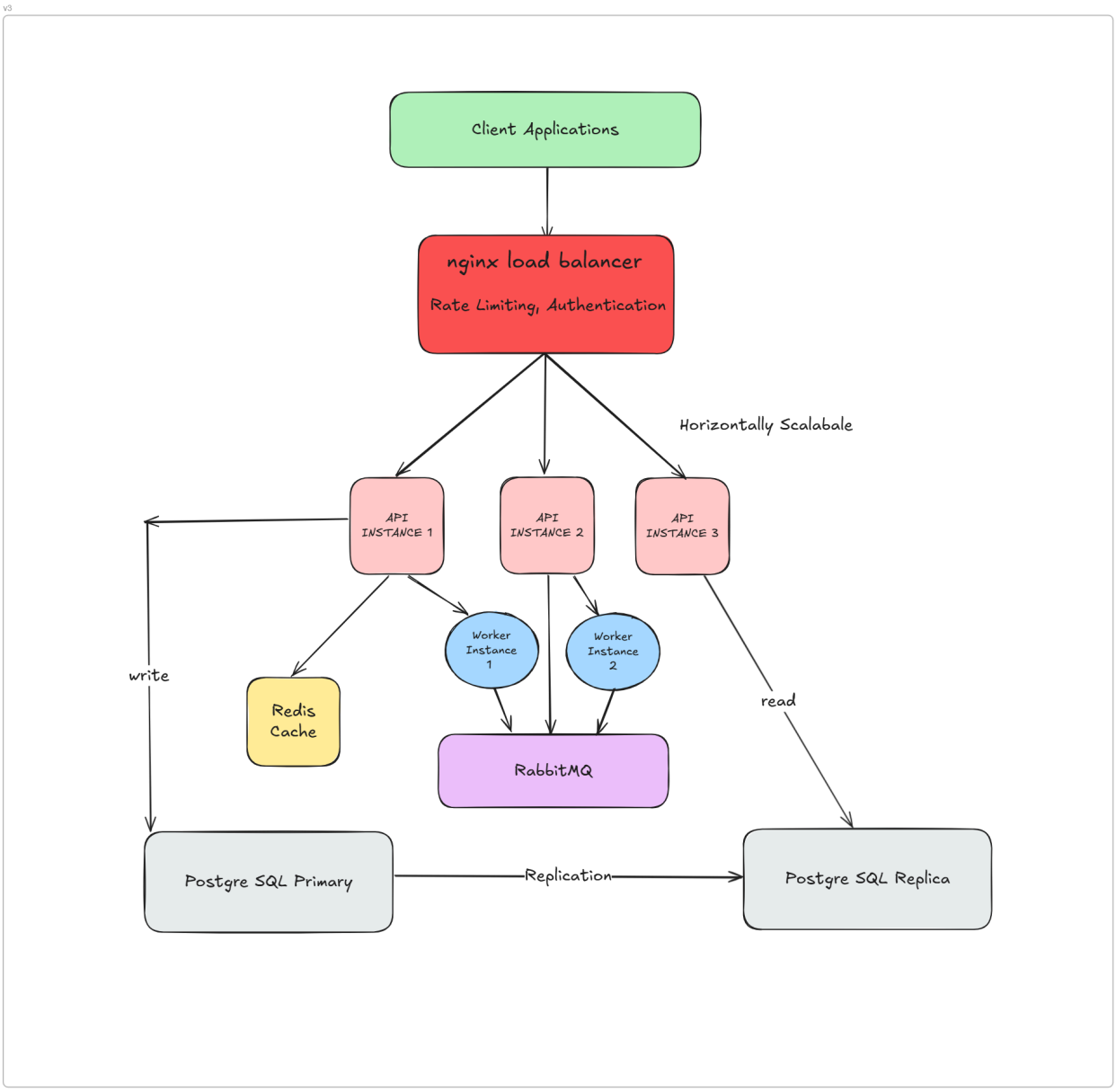
Architecture Modernization

- Moved from monolithic design to microservices
- Added event-driven communication between services
- Implemented infrastructure as code for consistent deployment

Advanced Features

- Added comprehensive reporting capabilities
- Implemented asynchronous report generation
- Enhanced audit logging for all inventory movements

System Architecture Diagram for v3:



Please find the [Github Repository] for complete code and further tests.